

Programming Practice - C++

Star Patterns:

1. Square

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Logic:

1. Take a number n (size of square).
2. Use an outer loop to control rows. (repeat n times).
3. Inside each row, use an inner loop to print stars (n times).
4. After printing one row, move to next line.

```
→ #include <iostream>
using namespace std;
```

```
int main() {
    int n = 5;
```

```
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            cout << " * ";
        }
        cout << endl;
    }
    return 0;
```

```
}
```

main.cpp



Share

Run

Output

Clear

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n = 5;
6
7     for(int i = 1; i <= n; i++)
8     {
9         for(int j = 1; j <= n; j++)
10        {
11            cout << "A";
12        }
13        cout << endl;
14    }
15    return 0;
16 }
```

```
*****
*****
*****
*****
*****
```

=== Code Execution Successful ===

2. Hollow Square

```
* * * * *
```

```
*       *
```

```
*       *
```

```
*       *
```

```
* * * * *
```

Logic:

1. Use two loops:

- Outer loop - controls row (i)
- inner loop - controls column (j)

2. Print * if it lies on boundary

- first row ($i=1$) & last row ($i=n$), first column ($j=1$) and last column ($j=n$)

```
→ #include <iostream>
using namespace std;
```

```
int main() {
    int n=5;
```

```
    for (int i=1; i<=n; i++)
```

```
    {
        for (int j=1; j<=n; j++)
```

```
        {
            if ( $i==1 || i==n || j==1 || j==n$ )
```

```
            {
                cout << "*";
```

```
            }
            else
```

```
            {
                cout << " ";
```

```
            }
            cout << endl;
```

```
        }
    }
    return 0;
```

```
}
```

main.cpp



Share

Run

Output

Clear

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n = 5;
6
7     for(int i = 1; i <= n; i++)
8     {
9         for(int j = 1; j <= n; j++)
10        {
11            if(i == 1 || i == n || j == 1 || j == n)
12            {
13                cout << "x";
14            }
15            else
16            {
17                cout << " ";
18            }
19        }
20        cout << endl;
21    }
22    return 0;
23 }
```

x x

x x

x x

=== Code Execution Successful ===

3. Triangle

```
*
**
***
****
*****
```

Logic:

1. Use an outer loop to control row (i).
2. Use inner loop to control column (j).
3. In every row, print stars equal to row number.
4. After printing, move to next line.

```
→ #include <iostream>
using namespace std;
```

```
int main() {
    int n = 5;
```

```
    for (int i = 1; i <= n; i++)
```

```
    {
        for (int j = 1; j <= i; j++)
```

```
        {
            cout << " ";
```

```
        }
        cout << endl;
```

```
    }
    return 0;
```

```
}
```

main.cpp



Share

Run

Output

Clear

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n = 5;
6
7     for(int i = 1; i <= n; i++)
8     {
9         for(int j = 1; j <= i; j++)
10        {
11            cout << "x";
12        }
13        cout << endl;
14    }
15    return 0;
16 }
```

```
*
**
***
****
*****
```

=== Code Execution Successful ===

4. Inverted triangle

```
* * * * *
* * * *
* * *
* *
*
```

Logic:

1. Use an outer loop to control row(i).
2. Start from n & decreases to 1.
3. Use an inner loop to print stars.
4. In each row, print stars equal to current row number(i).
5. After printing one row, move to next line.

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n = 5;
```

```
    for (int i = n; i >= 1; i--)
```

```
    {
        for (int j = 1; j <= i; j++)
```

```
        {
            cout << "*";
```

```
        }
        cout << endl;
```

```
    }
    return 0;
```

```
}
```

main.cpp

Share

Run

Output

Clear

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n = 5;
6
7     for(int i = n; i >= 1; i--)
8     {
9         for(int j = 1; j <= i; j++)
10         {
11             cout << " * ";
12         }
13         cout << endl;
14     }
15     return 0;
16 }
```

```
*****
*****
****
***
**
*

=== Code Execution Successful ===
```


5. Pyramid

```

  *
 * * *
* * * * *
* * * * * * *
* * * * * * * * *

```

Logic:

1. Use an outer loop to control rows
2. For each row: print spaces first & then print stars.
3. Number of spaces = $n - i$
4. Number of stars = $2 * i - 1$
5. After printing a row, move to next line

```

→ #include <iostream>
using namespace std;

```

```

int main() {
    int n = 5;

```

```

    for (int i = 1; i <= n; i++)
    {

```

```

        for (int j = 1; j <= n - i; j++)
        {

```

```

            cout << " ";

```

```

        }
        for (int j = 1; j <= 2 * i - 1; j++)
        {

```

```

            cout << "*";

```

```

        }
        cout << endl;

```

```

    }
    return 0;

```

```

}

```

main.cpp



Share

Run

Output

Clear

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n = 5;
6
7     for(int i = 1; i <= n; i++)
8     {
9
10        for(int j = 1; j <= n-i; j++)
11        {
12            cout << " ";
13        }
14        for(int j = 1; j <= 2*i-1; j++)
15        {
16            cout << "*";
17        }
18        cout << endl;
19    }
20    return 0;
21 }
```

```
*
***
*****
*****
*****
```

=== Code Execution Successful ===

6. Diamond

```

      *
     ***
    *****
   *********
  ***********
 *****
  ***
   *
  
```

Logic:

1. Upper half:

spaces = $n - i$

stars = $2 * i - 1$

Row 1 $\rightarrow$ 4 spaces, 1 star

Row 2 $\rightarrow$ 3 spaces, 3 stars

Row 3 $\rightarrow$ 2 spaces, 5 stars

Row 4 $\rightarrow$ 1 space, 7 stars

Row 5 $\rightarrow$ 0 spaces, 9 stars

2. Lower half - start from row $n - 1$ & decrease to 1.

```

→ #include <stdio.h>
#include <iostream>
using namespace std;
  
```

```
int main()
```

```
int n = 5;
```

```
for (int i = 1; i <= n; i++)
```

```
for (int j = 1; j <= n - i; j++)
```

```
cout << " ";
```

```
}
```

```
for (int j = 1; j <= 2 * i - 1; j++)
```

```
cout << "*";
```

```
}
```

```
cout << endl;
```

```
}
```

```
for (int i = n - 1; i >= 1; i--)
```

```
for (int j = 1; j <= n - i; j++)
```

```
cout << " ";
```

```
}
```

```
for (int j = 1; j <= 2 * i - 1; j++)
```

```
cout << "*";
```

```
}
```

```
cout << endl;
```

```
return 0;
```

main.cpp



Share

Run

Output

Clear

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     int n = 5;
5     for(int i = 1; i <= n; i++) {
6         for(int j = 1; j <= n-i; j++){
7             cout << " ";
8         }
9         for(int j = 1; j <= 2*i-1; j++){
10             cout << "*";
11         }
12         cout << endl;
13     }
14     for(int i = n-1; i >= 1; i--) {
15         for(int j = 1; j <= n-i; j++){
16             cout << " ";
17         }
18         for(int j = 1; j <= 2*i-1; j++){
19             cout << "*";
20         }
21         cout << endl;
22     }
23     return 0;
24 }
```

```
*
***
*****
*****
*****
***
*
```

=== Code Execution Successful ===

7. Half Diamond

```

*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*

```

Logic:

1. Upper half (Triangle)
 - No. of stars increase by 1.
 - Row 1 $\rightarrow$ 1 star
 - Row 2 $\rightarrow$ 2 star ---
 - Row 5 $\rightarrow$ 5 star
2. Lower half (Inverted triangle)
 - Start from row $n-1$.
 - No. of star decreases by 1.
 - Row 4 $\rightarrow$ 4 star
 - Row 3 $\rightarrow$ 3 star ---
 - Row 1 $\rightarrow$ 1 star

$\rightarrow$ #include <iostream>
using namespace std;

```

int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << " * ";
        }
        cout << endl;
    }
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            cout << " * ";
        }
        cout << endl;
    }
    return 0;
}

```


main.cpp



Share

Run

Output

Clear

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n = 5;
6
7     for(int i = 1; i <= n; i++)
8     {
9         for(int j = 1; j <= i; j++)
10        {
11            cout << "x";
12        }
13        cout << endl;
14    }
15    for(int i = n-1; i >= 1; i--)
16    {
17        for(int j = 1; j <= i; j++)
18        {
19            cout << "x";
20        }
21        cout << endl;
22    }
23    return 0;
24 }
```

```
*
**
***
****
*****
****
***
**
*
```

=== Code Execution Successful ===

8. Half Diamond Inverted

* * * * *

* * * *

* * *

* *

*

* *

* * *

* * * *

* * * * *

Logic:

1. Upper Half (Inverted triangle)
 - Start with n stars.
 - Decrease the no. of stars by 1 in each row.
 - Row 1 $\rightarrow$ 5 stars
 - Row 2 $\rightarrow$ 4 stars
 - Row 5 $\rightarrow$ 1 star.
2. Lower Half (triangle)
 - Start from 2 stars (to avoid repeating middle row).
 - Increase the no. of stars by 1.

$\rightarrow$ #include <iostream>
using namespace std;

int main()

{
int n = 5;

for (int i = n; i >= 1; i--) {

for (int j = 1; j <= i; j++) {

cout << " * ";

}

cout << endl;

}

for (int i = 2; i <= n; i++) {

for (int j = 1; j <= i; j++) {

cout << " * ";

}

cout << endl;

}

return 0;

}

main.cpp



Share

Run

Output

Clear

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n = 5;
6
7     for(int i = n; i >= 1; i--)
8     {
9         for(int j = 1; j <= i; j++)
10        {
11            cout << "*";
12        }
13        cout << endl;
14    }
15    for(int i = 2; i <= n; i++)
16    {
17        for(int j = 1; j <= i; j++)
18        {
19            cout << "*";
20        }
21        cout << endl;
22    }
23    return 0;
24 }
```

```
*****
****
***
**
*
**
***
****
*****
```

=== Code Execution Successful ===

Number Patterns:

1. Square

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

Logic:

1. Use an outer loop to control rows.
2. Use an inner loop to control columns.
3. Print the number 1 in every iteration of the inner loop.
5. After printing one row, move to the next line.

```
→ #include <std>
#include <iostream>
using namespace std;
```

```
int main() {
    int n = 5;

    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= n; j++)
        {
            cout << "1";
        }
        cout << endl;
    }
    return 0;
}
```

main.cpp

Share

Run

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n = 5;
6
7     for(int i = 1; i <= n; i++)
8     {
9         for(int j = 1; j <= n; j++)
10         {
11             cout << "1 ";
12         }
13         cout << endl;
14     }
15     return 0;
16 }
```

Output

Clear

```
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1

=== Code Execution Successful ===
```


2. Incrementing

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

1 2 3 4 5

Logic:

1. Use an outer loop to control rows.
2. Use an inner loop to control columns.
3. Print the value of the inner loop variable (j).
4. Since j starts from 1 & is incremented up to 5, no. are printed in increasing order.

```
→ #include <iostream>
using namespace std;
```

```
int main() {
```

```
    for (int i = 1; i <= 5; i++)
```

```
    {
        for (int j = 1; j <= 5; j++)
```

```
        {
            cout << j << " ";
```

```
        }
        cout << endl;
```

```
    }
    return 0;
```

```
}
```

main.cpp



Share

Run

Output

Clear

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5
6     for(int i = 1; i <= 5; i++)
7     {
8         for(int j = 1; j <= 5; j++)
9         {
10             cout << j << " ";
11         }
12         cout << endl;
13     }
14     return 0;
15 }
```

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

=== Code Execution Successful ===

3. Right Triangle

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Logic:

1. Use an outer loop to control row
2. Use an inner loop to control column
3. In every row, print numbers from 1 up to current row no.
4. After printing one row, move to next line.

```
→ #include <iostream>
using namespace std;

int main () {
```

```
    for (int i=1; i<=5; i++)
    {
        for (int j=1; j<=i; j++)
        {
            cout << j << " ";
        }
        cout << endl;
    }
    return 0;
```

```
}
```

main.cpp

Share

Run

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5
6     for(int i = 1; i <= 5; i++)
7     {
8         for(int j = 1; j <= i; j++)
9         {
10             cout << j << " ";
11         }
12         cout << endl;
13     }
14     return 0;
15 }
```

Output

Clear

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

=== Code Execution Successful ===

4. Inverted Diamond

3 3 3 3 3

4 4 4 4

5 5 5

6 6

7

Logic:

- 1. Number increase from 3 to 7
- Spaces increase by 1 in each row.
- Count of no.s decrease by 1 in each row.
- 3 → print 5 times
- 4 → print 4 times -- up to 7

```
→ #include <iostream>
using namespace std;
```

```
int main()
```

```
{
    for (int i = 3; i <= 7; i++)
```

```
{
    for (int j = 3; j < i; j++)
```

```
{
    cout << " ";
```

```
}
    for (int j = i; j <= 7; j++)
```

```
{
    cout << i << " ";
```

```
}
    cout << endl;
```

```
}
    return 0;
```

```
}
```


main.cpp



Share

Run

Output

Clear

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     for(int i = 3; i <= 7; i++)
5     {
6         for(int j = 3; j < i; j++)
7         {
8             cout << " ";
9         }
10        for(int j = i; j <= 7; j++)
11        {
12            cout << i << " ";
13        }
14        cout << endl;
15    }
16    return 0;
17 }
```

```
3 3 3 3 3
4 4 4 4
5 5 5
6 6
7

=== Code Execution Successful ===
```

5. Sandwich Pattern

3 3 3 3 3

4 4 4

5

4 4 4

3 3 3 3 3

Logic:

1. Upper half (3 to 5)

- Print leading spaces = $i-3$

- Print number i

- Number of times = $n-2*(i-3)$

2. Lower half (4 to 3)

- Print leading spaces = $i-3$

- Print number i

- Number of times = $n-2*(i-3)$

```
→ #include <iostream>
using namespace std;
```

```
int main() {
    int n = 5;
    for (int i = 3; i <= 5; i++) {
        for (int j = 0; j < i-3; j++) {
            cout << " ";
        }
        for (int j = 0; j < n-2*(i-3); j++) {
            cout << i << " ";
        }
        cout << endl;
    }
    for (int i = 4; i >= 3; i--) {
        for (int j = 0; j < i-3; j++) {
            cout << " ";
        }
        for (int j = 0; j < n-2*(i-3); j++) {
            cout << i << " ";
        }
        cout << endl;
    }
    return 0;
}
```

main.cpp				Share	Run	Output	Clear
<pre>1 #include <iostream> 2 using namespace std; 3 int main() { 4 int n = 5; 5 for(int i = 3; i <= 5; i++) { 6 for(int j = 0; j < i - 3; j++) { 7 cout << " "; 8 } 9 for(int j = 0; j < n - 2*(i-3); j++) { 10 cout << i << " "; 11 } 12 cout << endl; 13 } 14 for(int i = 4; i >= 3; i--) { 15 for(int j = 0; j < i - 3; j++) { 16 cout << " "; 17 } 18 for(int j = 0; j < n - 2*(i-3); j++) { 19 cout << i << " "; 20 } 21 cout << endl; 22 } 23 return 0; 24 }</pre>						<pre>3 3 3 3 3 4 4 4 5 4 4 4 3 3 3 3 3 === Code Execution Successful ===</pre>	

6. Incrementing Diamond Pattern

Logic:

3

4 4

5 5 5

6 6 6 6

7 7 7 7 7

6 6 6 6

5 5 5

4 4

3

1. Upper half

- start from 3 & go up to 7.
- print the current number.
- The count increases by 1 in each row.

2. Lower half

- start from 6 & go down to 3.
- print the current number repeatedly.
- The count decreases by 1 each row.

```
→ #include <iostream>
using namespace std;
```

```
int main () {
```

```
    for (int i = 3; i <= 7; i++)
```

```
    {
        for (int j = 3; j <= i; j++)
```

```
        {
            cout << i << " ";
```

```
        }
        cout << endl;
```

```
    }
    for (int i = 6; i >= 3; i--)
```

```
    {
        for (int j = 3; j <= i; j++)
```

```
        {
            cout << i << " ";
```

```
        }
        cout << endl;
```

```
    }
    return 0;
```

main.cpp

Share

Run

Output

Clear

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     for(int i = 3; i <= 7; i++)
6     {
7         for(int j = 3; j <= i; j++)
8         {
9             cout << i << " ";
10        }
11        cout << endl;
12    }
13    for(int i = 6; i >= 3; i--)
14    {
15        for(int j = 3; j <= i; j++)
16        {
17            cout << i << " ";
18        }
19        cout << endl;
20    }
21 }
22 return 0;
23 }
```

```
3
4 4
5 5 5
6 6 6 6
7 7 7 7 7
6 6 6 6
5 5 5
4 4
3

=== Code Execution Successful ===
```